

AUTHORIZATION PENTEST REPORT

Target: AuthLab Vulnerable Authorization Testing Application

Assessor: Jonathan Oyo

Date: 19th/April/2026

1. EXECUTIVE SUMMARY

This report presents the findings of a security assessment conducted on the AuthLab web application, with a focus on authorization controls.

The assessment identified several **critical vulnerabilities** in the application's access control mechanisms. These include broken horizontal and vertical access controls, improper session management, and insecure handling of user-supplied input. These flaws allow attackers to:

- Access other users' data without authorization
- Escalate privileges to administrator level
- Manipulate application behavior without validation
- Enumerate valid users within the system

If exploited, these vulnerabilities could lead to **unauthorized data exposure, account takeover, and full system compromise**.

Immediate remediation is strongly recommended to ensure proper enforcement of access control policies and to protect system integrity.

2. SCOPE AND OBJECTIVES

Scope

The assessment focused on the **authorization mechanisms** of the AuthLab application hosted locally using XAMPP.

Objectives

- Evaluate enforcement of access control policies
- Identify authorization vulnerabilities

- Assess risks associated with improper access restrictions
- Provide actionable remediation recommendations

3. METHODOLOGY

The assessment followed a structured authorization testing approach:

1. **Surface Mapping**
Identification of accessible endpoints and functionalities.
2. **Object Identifier Analysis**
Examination of how user and resource identifiers are used.
3. **Horizontal Access Control Testing**
Testing access between users with the same privilege level.
4. **Vertical Access Control Testing**
Attempting privilege escalation to higher roles.
5. **Action-Level Control Testing**
Sending unexpected or malformed inputs to evaluate validation.
6. **Response Consistency Analysis**
Observing how the system responds to valid and invalid requests.

4. TOOLS USED

- **Burp Suite (Community Edition)** – Interception and manipulation of HTTP requests and responses
- **Chromium Browser** – Interaction with the application interface
- **XAMPP Server** – Local hosting environment for the application

5. FINDINGS

Finding 1: BROKEN SESSION MANAGEMENT

Severity: High

Description

The application issues a session identifier (PHPSESSID) before authentication and does not regenerate it after successful login.

Evidence

The same session ID is used before and after authentication:

1	http://localhost	GET	/authlab/pages/login.php	200	592	HTML	php	127...	PHPSESSID=3ijdmmc9mq879i6p9f78h96gkh
---	------------------	-----	--------------------------	-----	-----	------	-----	--------	--------------------------------------

PHPSESSID=3ijdmmc9mq879i6p9f78h96gkh

Impact

- Enables **session fixation attacks**
- Allows attackers to hijack authenticated sessions
- Compromises user account security

Recommendation

- Regenerate session IDs upon successful authentication
- Implement secure session handling mechanisms
- Set session cookies with HttpOnly and Secure flags

Finding 2: BROKEN HORIZONTAL ACCESS CONTROL (IDOR)

Severity: High

Description

The application allows users to access other users' data by modifying the id parameter in requests.

Evidence

Request	Response
<pre>1 GET /authlab/pages/profile.php?id=2 HTTP/1.1 2 Host: localhost 3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146" 4 sec-ch-ua-mobile: ?0 5 sec-ch-ua-platform: "Windows" 6 Accept-Language: en-US,en;q=0.9 7 Upgrade-Insecure-Requests: 1 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif ,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b 3;q=0.7 10 Sec-Fetch-Site: same-origin 11 Sec-Fetch-Mode: navigate 12 Sec-Fetch-User: ?1 13 Sec-Fetch-Dest: document 14 Referer: http://localhost/authlab/pages/dashboard.php 15 Accept-Encoding: gzip, deflate, br 16 Cookie: security=low; theme=dark; PHPSESSID= 3ijdmmc9mg879i6p9f78h96grh 17 Connection: keep-alive 18</pre>	Profile Username: student2 Role: student

GET /authlab/pages/profile.php?id=1

Changed to:

GET /authlab/pages/profile.php?id=2

Impact

- Unauthorized access to other users' sensitive data
- Violation of data confidentiality

Recommendation

- Enforce server-side authorization checks
- Ensure users can only access resources they own
- Avoid relying on client-supplied identifiers

Finding 3: BROKEN VERTICAL ACCESS CONTROL

Severity: High

Description

A normal user can access administrative functionality without proper authorization checks.

Evidence

The screenshot displays the network tab of a web browser's developer tools, showing an HTTP request and its corresponding response.

Request:

- Method: GET
- URL: /authlab/pages/admin.php
- Host: localhost
- sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
- sec-ch-ua-mobile: ?0
- sec-ch-ua-platform: "Windows"
- Accept-Language: en-US,en;q=0.9
- Upgrade-Insecure-Requests: 1
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
- Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
- Sec-Fetch-Site: same-origin
- Sec-Fetch-Mode: navigate
- Sec-Fetch-User: ?1
- Sec-Fetch-Dest: document
- Referer: http://localhost/authlab/pages/dashboard.php
- Accept-Encoding: gzip, deflate, br
- Cookie: security=low; theme=dark; PHPSESSID=3ijdmac9mg879i6p9f78h96gkh
- Connection: keep-alive

Response:

- Status: 200 OK
- Date: Fri, 17 Apr 2026 23:24:50 GMT
- Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
- X-Powered-By: PHP/8.2.12
- Expires: Thu, 19 Nov 1981 08:52:00 GMT
- Cache-Control: no-store, no-cache, must-revalidate
- Pragma: no-cache
- Content-Length: 117
- Keep-Alive: timeout=5, max=100
- Connection: Keep-Alive
- Content-Type: text/html; charset=UTF-8

The response body contains the following HTML:

```
<h2>
  Admin Panel
</h2>
<!-- NO ROLE CHECK -->
<a href="update_role.php?id=1&role=admin">
  Make User Admin
</a>
```

A user with student privileges successfully accessed the admin panel.

Impact

- Unauthorized access to administrative features
- Potential full system compromise

Recommendation

- Implement strict role-based access control (RBAC)
- Validate user roles on every request

- Restrict access to sensitive endpoints

Finding 4: PRIVILEGE ESCALATION VIA PARAMETER MANIPULATION

Severity: Critical

Description

The application allows users to escalate privileges by modifying request parameters.

Evidence

Request	Response
PrettyRawHex 1 GET /authlab/pages/update_role.php?id=1&role=admin HTTP/1.1 2 Host: localhost 3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146" 4 sec-ch-ua-mobile: ?0 5 sec-ch-ua-platform: "Windows" 6 Accept-Language: en-US,en;q=0.9 7 Upgrade-Insecure-Requests: 1 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 9 Accept: 10 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w 11 ebp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: navigate 14 Sec-Fetch-User: ?1 15 Sec-Fetch-Dest: document 16 Referer: http://localhost/authlab/pages/admin.php 17 Accept-Encoding: gzip, deflate, br 18 Cookie: security=low; theme=dark; PHPSESSID=3ijdmcmc9mg879i6p9f78h96gkrlk 19 Connection: keep-alive 20	PrettyRawHexRender Role updated

role=student → role=admin

Impact

- Attackers can gain administrative privileges
- Full compromise of application functionality

Root Cause

The server trusts client-supplied input for authorization decisions.

Recommendation

- Do not trust client-side input for role assignment
- Enforce role validation on the server side
- Implement backend authorization logic independent of user input

Finding 5: MISSING INPUT VALIDATION

Severity: Medium

Description

The application accepts invalid inputs (e.g., empty role values, using a completely non existing role like driver) and still processes them successfully.

Evidence

Request		Response
Pretty Raw Hex 1 GET /authlab/pages/update_role.php?id=1&role= HTTP/1.1 2 Host: localhost		Pretty Raw Hex Render Role updated

- Empty role submitted

- System responds: “Role updated” and role is updated indeed:

Profile

Username: student1

Role:

Impact

- Enables blind exploitation
- Facilitates automated attacks
- Leads to inconsistent application state

Recommendation

- Implement strict input validation
- Reject invalid or malformed requests
- Use allowlists for acceptable values

Finding 6: PREDICTABLE USER IDENTIFIERS (USER ENUMERATION)

Severity: Low

Description

User accounts use sequential numeric identifiers, allowing enumeration.

Evidence

6. Intruder attack of http://localhost

Attack ▾

Results Positions

Capture filter: Capturing all items ☐ Apply

View filter: Showing all items

Request	Payload	Status code ^	Response r...	Error	Timeout	Length	>Username:	>Role:	Comment
0		200	168			432	student1		
1	1	200	3			431	student1		
2	2	200	5			439	student2	student	
3	3	200	10			440	lecturer1	lecturer	
4	4	200	3			435	admin1	admin	
5	5	200	4			728	 Warning: ... Warning: ...		
6	6	200	5			729	 Warning: ... Warning: ...		
7	7	200	14			728	 Warning: ... Warning: ...		
8	8	200	7			729	 Warning: ... Warning: ...		
9	9	200	19			728	 Warning: ... Warning: ...		
10	10	200	18			729	 Warning: ... Warning: ...		

Exiting users

Automated requests from ID 1–10 revealed valid users:

- IDs 1–4 exist

Impact

- Enables automated data extraction
- Assists attackers in targeted attacks

Recommendation

- Use non-sequential, random identifiers (e.g., UUIDs)
- Implement rate limiting to prevent enumeration

6. CONCLUSION

The assessment revealed **critical weaknesses in the application's authorization mechanisms**, particularly in access control enforcement and session handling.

The most severe issues—**privilege escalation and broken access controls**—allow attackers to gain unauthorized access and potentially take full control of the system.

To improve the application's security posture, it is essential to:

- Enforce strict server-side authorization checks
- Implement proper session management
- Validate all user inputs
- Adopt secure design principles for access control

Addressing these vulnerabilities will significantly reduce the risk of exploitation and improve overall system security.